

Boolean functions:

applications in quantum error correction and beyond

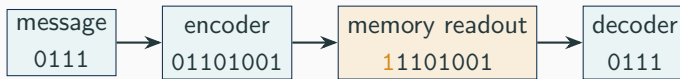
Bohan Lu Duke University

One worked example: encode four bits, correct one error, then use the same functions to calculate a quantum gate.

Classical encoding Quantum construction Further applications

Correcting a bit error in memory

Error-correcting memory stores extra bits so a changed bit can be recovered.



The *encoder* computes the stored bits from the message. The *decoder* reconstructs the message from the values read back.

Assumption: at most one stored bit changes. The decoder does not know which position changed.

We will specify an eight-bit example and prove that its decoder corrects any one changed bit.

Memory application: IBM, Troubleshooting Memory issues. The eight-bit example is a teaching construction.

From four message bits to eight stored values

XOR returns 1 exactly when its two bits differ. Write it as $\oplus$.

input pair	00	01	10	11
XOR	0	1	1	0

The message is four bits (b, a_1, a_2, a_3) . A position is a three-bit address $u = (u_1, u_2, u_3)$. At address u , start with b and XOR each u_i for which $a_i = 1$.

For message $(0, 1, 1, 1)$, the stored bit is the *parity*, the XOR of the three address bits:

$$p(u) = u_1 \oplus u_2 \oplus u_3, \quad p(101) = 1 \oplus 0 \oplus 1 = 0.$$

address u	000	001	010	011	100	101	110	111
stored $p(u)$	0	1	1	0	1	0	0	1

The three address bits locate a stored value. The four message bits specify all eight stored values.

Affine Boolean functions

A *Boolean function* assigns a bit to every binary address, as in the preceding table. Write $\mathbb{F}_2 = \{0, 1\}$ with XOR addition.

The message coefficients define the *affine Boolean function*

$$f_{a,b}(u) = \underbrace{b}_{\text{constant}} \oplus \underbrace{a_1 u_1 \oplus a_2 u_2 \oplus a_3 u_3}_{\text{selected coordinates}}, \quad a = (a_1, a_2, a_3).$$

A product $a_i u_i$ is u_i when $a_i = 1$, and 0 when $a_i = 0$.

$$f_{100,0}(u) = u_1 \longleftrightarrow 00001111, \quad f_{000,1}(u) = 1 \longleftrightarrow 11111111.$$

An allowed stored word is a *codeword*. The *code* consists of the 16 evaluation words, one for each four-bit message.

The decoder knows this encoder, so it knows exactly which 16 stored words are allowed.

From a truth table to a polynomial

The *algebraic normal form* (ANF) is the unique XOR of products of distinct variables representing a Boolean function. Products are AND: $u_1 u_2 = 1$ only when both bits are 1. The *degree* is the largest number of variables in a nonzero product.

Suppose the value at 000 changes. Call the received table r . The *indicator* of 000 is 1 there and 0 at all other addresses:

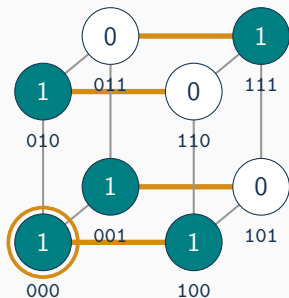
$$\delta_{000}(u) = (1 \oplus u_1)(1 \oplus u_2)(1 \oplus u_3).$$

Each factor tests whether one address bit is zero. Their product tests all three.

$$\begin{aligned} r &= p \oplus \delta_{000} \\ &= 1 \oplus u_1 u_2 \oplus u_1 u_3 \oplus u_2 u_3 \oplus u_1 u_2 u_3. \end{aligned}$$

Every allowed function has degree at most one. The received function has degree three, so its table is not an allowed codeword.

Comparing neighboring table entries



Below: address. Teal: 1. White: 0.
Gold: first-bit edges. Ring: changed entry.

The direction $e_1 = (1, 0, 0)$ toggles the first address bit. Vector XOR acts coordinate by coordinate.

$$r(000) \oplus r(100) = 1 \oplus 1 = 0.$$

The *Boolean derivative* records this comparison at each address:

$$D_{e_i} f(u) = f(u \oplus e_i) \oplus f(u),$$

where e_i has a 1 in position i and zeros elsewhere.

Derivatives of p and r along e_1 :

$$D_{e_1} p = 1, \quad D_{e_1} r = 1 \oplus (1 \oplus u_2)(1 \oplus u_3).$$

One changed vertex reverses exactly one of four disjoint edge comparisons in each fixed direction.

Recovering the four message coefficients

How can we recover a_i ? Toggling u_i changes only the term $a_i u_i$:

$$D_{e_i} f_{a,b} = a_i.$$

The four disjoint e_1 edges therefore give four estimates of a_1 .

addresses	000, 100	001, 101	010, 110	011, 111
received XOR	0	1	1	1

At most one estimate is wrong, because the readout has at most one changed entry.

Recovering the four message coefficients

How can we recover a_i ? Toggling u_i changes only the term $a_i u_i$:

$$D_{e_i} f_{a,b} = a_i.$$

The four disjoint e_1 edges therefore give four estimates of a_1 .

addresses	000, 100	001, 101	010, 110	011, 111
received XOR	0	1	1	1

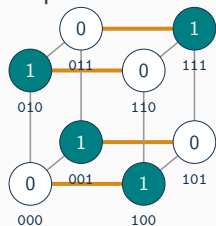
At most one estimate is wrong, because the readout has at most one changed entry. Majority gives $a_1 = 1$. The other two directions give $a_2 = a_3 = 1$.

$$r(u) \oplus a_1 u_1 \oplus a_2 u_2 \oplus a_3 u_3 \longleftrightarrow 10000000.$$

The remaining seven zeros recover $b = 0$. The decoded message is $(0, 1, 1, 1)$.

Why distinct codewords differ in at least four positions

The *weight* $\text{wt}(f)$ counts the ones in the evaluation word. The *Hamming distance* $d(f, g)$ counts positions where two words differ.



Below: u . Circle: $p(u)$. Teal 1, white 0. Gold pairs.

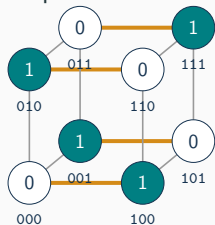
Each pair has one zero and one one, so $\text{wt}(p) = 4$. For any nonconstant affine function, pair across a coordinate with coefficient 1.

$$H = \{u : p(u) = 1\} = \{100, 010, 001, 111\}.$$

Such a level set is called an *affine hyperplane*.

Why distinct codewords differ in at least four positions

The *weight* $\text{wt}(f)$ counts the ones in the evaluation word. The *Hamming distance* $d(f, g)$ counts positions where two words differ.



Each pair has one zero and one one, so $\text{wt}(p) = 4$. For any nonconstant affine function, pair across a coordinate with coefficient 1.

$$H = \{u : p(u) = 1\} = \{100, 010, 001, 111\}.$$

Such a level set is called an *affine hyperplane*.

Below: u . Circle: $p(u)$. Teal 1,
white 0. Gold pairs.

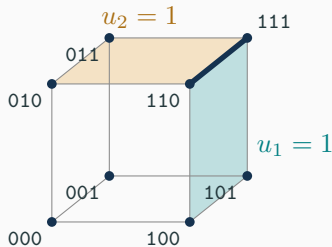
For distinct affine f, g , their XOR is either nonconstant or constant one:

$$d(f, g) = \text{wt}(f \oplus g) = 4 \text{ or } 8, \quad \text{respectively.}$$

Two codewords each one flip from r would differ in at most two places. Distance at least four makes one-error recovery unique.

Counting intersections of Boolean supports

The *support* $\text{supp}(f)$ is the set of addresses where f has value 1. For functions f, g , the product fg selects their common support.



Dots: addresses. Teal: $u_1 = 1$. Gold:
 $u_2 = 1$. Dark edge: both equal 1.

Ordinary addition counts a common 1 twice. XOR counts it zero times:

$$\text{wt}(f \oplus g) = \text{wt}(f) + \text{wt}(g) - 2 \text{wt}(fg).$$

function	weight	support
u_1	4	teal face
$u_1 u_2$	2	dark common edge
$u_1 u_2 u_3$	1	endpoint 111

Single-qubit rotations produce phases

A qubit is a unit vector in $\mathbb{C}^2$. Its computational basis vectors are $|0\rangle = (1, 0)^T$ and $|1\rangle = (0, 1)^T$. A *relative phase* multiplies one component by $e^{i\theta}$ for a real angle θ :

$$\frac{|0\rangle + |1\rangle}{\sqrt{2}} \mapsto \frac{|0\rangle + e^{i\theta}|1\rangle}{\sqrt{2}}.$$

For $\theta = \pi/4$, write $\omega = e^{i\pi/4}$ and call the rotation T . Its inverse $T^\dagger$ uses ω^{-1} . For an n -bit physical word v , $|v\rangle$ is its basis vector. Applying T separately at all n sites, written $T^{\otimes n}$, multiplies one factor ω for each 1:

$$T = \text{diag}(1, \omega), \quad T^{\otimes n}|v\rangle = \omega^{\text{wt}(v)}|v\rangle.$$

On three qubits we want only the basis vector $|111\rangle$ to change sign. This gate is *controlled-controlled-Z* (CCZ):

$$\text{CCZ}|a_1a_2a_3\rangle = (-1)^{a_1a_2a_3}|a_1a_2a_3\rangle.$$

Encoding three qubits with complementary words

Keep the three-bit label a . Pair the words with $b = 0$ and $b = 1$:

label a	$f_{a,0}$	$f_{a,1}$
000	00000000	11111111
100	00001111	11110000
111	01101001	10010110

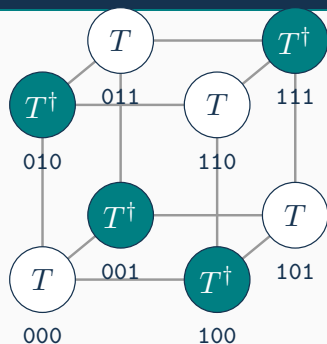
The corresponding *encoded basis state* is the normalized superposition

$$|a\rangle_L = \frac{|f_{a,0}\rangle + |f_{a,1}\rangle}{\sqrt{2}}.$$

Eight disjoint pairs give eight orthogonal states, encoding three logical qubits in eight physical qubits. A logical gate acts on these encoded states.

An operation that only multiplies physical basis vectors by phases preserves this encoding only if both words in each pair receive the same phase.

Choosing T and $T^\dagger$ with parity



Below: physical address u . White: $p(u) = 0$, apply T . Teal: $p(u) = 1$, apply $T^\dagger$.

The gate pattern depends on the physical address. Its action on an encoded state still needs to be checked.

The word $v = p$ has its four ones at $T^\dagger$ sites: its total rotation angle is $-4(\pi/4)$.

For any word v , count its ones at T sites positively and its ones at $T^\dagger$ sites negatively. This is its *signed weight*:

$$\text{sw}_p(v) = \sum_{u \in \mathbb{F}_2^3} (-1)^{p(u)} v(u).$$

The sum is an ordinary integer. The physical phase is $\omega^{\text{sw}_p(v)}$.

Checking the eight-qubit logical gate

At one address, $p = 0$ contributes v , while $p = 1$ contributes $-v$. Both cases give the integer identity

$$(-1)^{p(u)}v(u) = (p \oplus v)(u) - p(u).$$

Summing over addresses gives $\text{sw}_p(v) = \text{wt}(p \oplus v) - 4$. For $a \neq 111$, $p \oplus f_{a,b}$ is nonconstant affine, hence has weight four.

logical label a	$\text{sw}_p(f_{a,0})$	$\text{sw}_p(f_{a,1})$	phase
$a \neq 111$	0	0	
$a = 111$	-4	+4	

Checking the eight-qubit logical gate

At one address, $p = 0$ contributes v , while $p = 1$ contributes $-v$. Both cases give the integer identity

$$(-1)^{p(u)}v(u) = (p \oplus v)(u) - p(u).$$

Summing over addresses gives $\text{sw}_p(v) = \text{wt}(p \oplus v) - 4$. For $a \neq 111$, $p \oplus f_{a,b}$ is nonconstant affine, hence has weight four.

logical label a	$\text{sw}_p(f_{a,0})$	$\text{sw}_p(f_{a,1})$	phase
$a \neq 111$	0	0	+1
$a = 111$	-4	+4	-1

$\omega^4 = \omega^{-4} = -1$. Both words in every pair receive the same phase, and exactly the logical label 111 changes sign: CCZ.

What additional error protection requires

A bit error X swaps $|0\rangle, |1\rangle$. A phase error Z sends $|1\rangle$ to $-|1\rangle$ and fixes $|0\rangle$. A *stabilizer check* is an observable with value $+1$ on every encoded state.

Example: flipping all eight bits exchanges the two words in each pair, so $X^{\otimes 8}$ is a check. A single-site Z changes its outcome to -1 .

The *syndrome* lists check outcomes as bits: 0 for $+1$, 1 for -1 . For checks that flip selected sites, a single-site phase error has a column of responses:

$$\text{columns } \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ 1 \end{pmatrix} \implies \text{first two errors give } \begin{pmatrix} 1 \\ 0 \end{pmatrix} \oplus \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

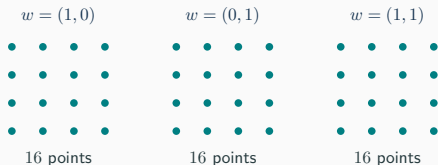
A zero column hides one phase error. Equal columns cancel, hiding two. Nonzero, distinct columns exclude both cases.

Quantum distance is the fewest sites on which a product of bit and phase errors can act undetected and change encoded information. The eight-qubit code has distance two. Distance three corrects an arbitrary one-qubit error.

Deleting a subspace leaves weights 24 and 32

Six-bit addresses have parts $u \in \mathbb{F}_2^4$ and $w \in \mathbb{F}_2^2$. Delete the subspace $w = 00$, retaining

$$\mathcal{P} = \{(u, w) : w \neq 00\}.$$



64 points – 16 deleted = 48.

Each dot is one address. Each block contains all 16 choices of u .

For coefficients $\alpha \in \mathbb{F}_2^4$, $\beta \in \mathbb{F}_2^2$, use binary dot products:

$$\alpha \cdot u = \alpha_1 u_1 \oplus \cdots \oplus \alpha_4 u_4$$

and similarly for $\beta \cdot w$. Define

$$\ell(u, w) = \alpha \cdot u \oplus \beta \cdot w.$$

Every nonzero ℓ has 32 ones on all 64 sites. On the deleted subspace, $\ell = \alpha \cdot u$: eight ones if $\alpha \neq 0$, zero otherwise. On retained sites, written $\ell|_{\mathcal{P}}$,

$$\text{wt}(\ell|_{\mathcal{P}}) = \begin{cases} 32 - 8 = 24, & \alpha \neq 0, \\ 32 - 0 = 32, & \alpha = 0, \beta \neq 0. \end{cases}$$

Completing the 48-qubit code

The six address coordinates give words $S_1, \dots, S_6$. Flipping their selected sites gives stabilizer checks. Their XOR combinations form a space S of 64 words.

Three logical functions give words K_1, K_2, K_3 . Let $K(a) = a_1 K_1 \oplus a_2 K_2 \oplus a_3 K_3$ for the logical label a .

Add each stabilizer word to the logical word:

$$|a\rangle_L = \frac{1}{8} \sum_{s \in S} |K(a) \oplus s\rangle.$$

The sign function γ selects T at value 0, $T^\dagger$ at value 1.

$w = (1,0)$	$w = (0,1)$	$w = (1,1)$																																																
<table> <tr><td>T</td><td>T</td><td>T</td><td>T</td></tr> <tr><td>$T^\dagger$</td><td>T</td><td>$T^\dagger$</td><td>T</td></tr> <tr><td>T</td><td>$T^\dagger$</td><td>$T^\dagger$</td><td>T</td></tr> <tr><td>$T^\dagger$</td><td>$T^\dagger$</td><td>T</td><td>T</td></tr> </table>	T	T	T	T	$T^\dagger$	T	$T^\dagger$	T	T	$T^\dagger$	$T^\dagger$	T	$T^\dagger$	$T^\dagger$	T	T	<table> <tr><td>T</td><td>$T^\dagger$</td><td>T</td><td>T</td></tr> <tr><td>T</td><td>$T^\dagger$</td><td>T</td><td>T</td></tr> <tr><td>$T^\dagger$</td><td>$T^\dagger$</td><td>T</td><td>$T^\dagger$</td></tr> <tr><td>T</td><td>T</td><td>$T^\dagger$</td><td>T</td></tr> </table>	T	$T^\dagger$	T	T	T	$T^\dagger$	T	T	$T^\dagger$	$T^\dagger$	T	$T^\dagger$	T	T	$T^\dagger$	T	<table> <tr><td>T</td><td>$T^\dagger$</td><td>$T^\dagger$</td><td>$T^\dagger$</td></tr> <tr><td>$T^\dagger$</td><td>$T^\dagger$</td><td>T</td><td>$T^\dagger$</td></tr> <tr><td>$T^\dagger$</td><td>T</td><td>T</td><td>T</td></tr> <tr><td>$T^\dagger$</td><td>$T^\dagger$</td><td>T</td><td>$T^\dagger$</td></tr> </table>	T	$T^\dagger$	$T^\dagger$	$T^\dagger$	$T^\dagger$	$T^\dagger$	T	$T^\dagger$	$T^\dagger$	T	T	T	$T^\dagger$	$T^\dagger$	T	$T^\dagger$
T	T	T	T																																															
$T^\dagger$	T	$T^\dagger$	T																																															
T	$T^\dagger$	$T^\dagger$	T																																															
$T^\dagger$	$T^\dagger$	T	T																																															
T	$T^\dagger$	T	T																																															
T	$T^\dagger$	T	T																																															
$T^\dagger$	$T^\dagger$	T	$T^\dagger$																																															
T	T	$T^\dagger$	T																																															
T	$T^\dagger$	$T^\dagger$	$T^\dagger$																																															
$T^\dagger$	$T^\dagger$	T	$T^\dagger$																																															
$T^\dagger$	T	T	T																																															
$T^\dagger$	$T^\dagger$	T	$T^\dagger$																																															
$6\ T^\dagger$	$6\ T^\dagger$	$10\ T^\dagger$																																																

Actual signs: gold T , teal $T^\dagger$. Rows $u_1 u_2$, columns $u_3 u_4$, ordered 00, 01, 11, 10.

The site geometry, logical functions, and rotation signs have separate roles. All three specify the construction.

Checking phases through row intersections

A *generator row* is one of the nine evaluation words $K_1, K_2, K_3, S_1, \dots, S_6$. For distinct rows g, h, q , products mean coordinatewise AND, hence intersect supports. The integer expansion at each site is

$$g \oplus h \oplus q = g + h + q - 2(gh + gq + hq) + 4ghq.$$

For a word v , set $\text{sw}_\gamma(v) = \sum_{j=1}^{48} (-1)^{\gamma_j} v_j$. Congruence modulo m means equality of integer remainders after division by m .

$$\text{sw}_\gamma(g) \equiv 0 \pmod{8} \qquad \text{coefficient 1,}$$

Checking phases through row intersections

A *generator row* is one of the nine evaluation words $K_1, K_2, K_3, S_1, \dots, S_6$. For distinct rows g, h, q , products mean coordinatewise AND, hence intersect supports. The integer expansion at each site is

$$g \oplus h \oplus q = g + h + q - 2(gh + gq + hq) + 4ghq.$$

For a word v , set $\text{sw}_\gamma(v) = \sum_{j=1}^{48} (-1)^{\gamma_j} v_j$. Congruence modulo m means equality of integer remainders after division by m .

$$\begin{array}{ll} \text{sw}_\gamma(g) \equiv 0 \pmod{8} & \text{coefficient } 1, \\ \text{sw}_\gamma(gh) \equiv 0 \pmod{4} & \text{coefficient } -2, \end{array}$$

Checking phases through row intersections

A *generator row* is one of the nine evaluation words $K_1, K_2, K_3, S_1, \dots, S_6$. For distinct rows g, h, q , products mean coordinatewise AND, hence intersect supports. The integer expansion at each site is

$$g \oplus h \oplus q = g + h + q - 2(gh + gq + hq) + 4ghq.$$

For a word v , set $\text{sw}_\gamma(v) = \sum_{j=1}^{48} (-1)^{\gamma_j} v_j$. Congruence modulo m means equality of integer remainders after division by m .

$$\text{sw}_\gamma(g) \equiv 0 \pmod{8} \qquad \text{coefficient } 1,$$

$$\text{sw}_\gamma(gh) \equiv 0 \pmod{4} \qquad \text{coefficient } -2,$$

$$\text{sw}_\gamma(ghq) \equiv \begin{cases} 1 & \pmod{2}, \quad \{g, h, q\} = \{K_1, K_2, K_3\}, \\ 0 & \pmod{2}, \quad \text{otherwise.} \end{cases}$$

Four or more rows have coefficients divisible by eight. Only the logical triple contributes $4a_1a_2a_3$ modulo eight, producing CCZ.

The resulting quantum code

The paper constructs an encoding with

$$\underbrace{48}_{\text{physical qubits}}, \quad \underbrace{3}_{\text{logical qubits}}, \quad \underbrace{3}_{\text{quantum distance}}.$$

The standard notation for these three quantities is $[[48, 3, 3]]$.

Error correction: the code corrects any error on one physical qubit.

Logical operation: the displayed $26T + 22T^\dagger$ pattern implements CCZ, with equal phase on all 64 words in each encoded basis state.

The explicit functions give one construction. The signed-overlap conditions test phase correctness for any code specified by such rows and signs.

Current paper: fixed logical labels, explicit sign pattern, and signed-overlap criterion.

Reed–Muller codes and finite-length bounds

Our classical code evaluates affine functions on three-bit addresses. It is the *Reed–Muller code* $\text{RM}(1, 3)$. More generally, $\text{RM}(d, m)$ evaluates degree-at-most- d Boolean polynomials on all m -bit addresses.

A binary code closed under XOR is *linear*. Its parameters $[n, k, d_{\min}]$ mean n stored bits, k message bits, and minimum distance $d_{\min}$. Our code is $[8, 4, 4]$.

Each codeword has 9 possible readouts with at most one error: itself and its 8 single-bit changes. Unique correction requires these sets to be disjoint.

$$\underbrace{16}_{\text{codewords}} \underbrace{(1 + 8)}_{\text{readouts per codeword}} = 144 \leq \underbrace{2^8}_{\text{all readouts}} = 256.$$

For M length- n codewords correcting one error, the same count gives $M(1 + n) \leq 2^n$, the one-error *Hamming bound*.

The bound limits the number of messages at a fixed length under a worst-case error guarantee.

Erasure recovery and channel capacity

An *erasure* is a missing value with a known position, shown by ?. The same code gives two different examples:

readout	explanation
?1101001	only parity's codeword fits the seven known values
0??0?00?	both 00000000 and 01101001 fit

Now erase each position independently with probability ε . The *rate* is message bits per stored bit, $R = k/n$, here $4/8 = 1/2$. *Capacity* is the supremum of rates allowing error probability to approach zero as length grows.

Erasure recovery and channel capacity

An *erasure* is a missing value with a known position, shown by ?. The same code gives two different examples:

readout	explanation
?1101001	only parity's codeword fits the seven known values
0??0?00?	both 00000000 and 01101001 fit

Now erase each position independently with probability ε . The *rate* is message bits per stored bit, $R = k/n$, here $4/8 = 1/2$. *Capacity* is the supremum of rates allowing error probability to approach zero as length grows. For this binary erasure channel, $C = 1 - \varepsilon$. For example, at $\varepsilon = 1/4$, rates below $3/4$ can be made reliable.

Optimal erasure decoding finds the unique compatible codeword, or reports ambiguity. For every target $R \in (0, 1)$, a sequence of Reed–Muller codes has lengths growing, rates tending to R , and whole-word recovery probability tending to one when $\varepsilon < 1 - R$.

Kudekar et al., Reed–Muller Codes Achieve Capacity on Erasure Channels, Theorem 27.

Predicting a Boolean output with parity

How often does parity predict this table? Choose each three-bit input with probability $1/8$ (uniform sampling).

u	000	001	010	011	100	101	110	111
$r(u)$	1	1	1	0	1	0	0	1
$p(u)$	0	1	1	0	1	0	0	1

Seven agreements, one disagreement: prediction probability $7/8$.

Predicting a Boolean output with parity

How often does parity predict this table? Choose each three-bit input with probability $1/8$ (uniform sampling).

u	000	001	010	011	100	101	110	111
$r(u)$	1	1	1	0	1	0	0	1
$p(u)$	0	1	1	0	1	0	0	1

Seven agreements, one disagreement: prediction probability $7/8$. Encode a bit z by $(-1)^z$: $0 \mapsto +1$, $1 \mapsto -1$. The average sign product is a normalized *Fourier coefficient* of $(-1)^r$, at parity p :

$$\frac{1}{8} \sum_{u \in \mathbb{F}_2^3} (-1)^{r(u)} (-1)^{p(u)} = \frac{7-1}{8} = \frac{3}{4}, \quad \Pr[r = p] = \frac{1 + 3/4}{2} = \frac{7}{8}.$$

A cryptographic Boolean component computes one output bit from input bits. Accurate affine prediction is undesirable. This degree-three table still agrees with parity on $7/8$ of inputs.

Explicit construction. Forty-eight physical qubits encode three logical qubits and correct any one-qubit error.

Physical implementation. Twenty-six T and twenty-two $T^\dagger$ rotations implement logical CCZ without an additional logical correction.

Exact verification. Signed weights must give one phase throughout each encoded superposition. The row-intersection conditions verify this requirement.

In the worked example, affine-function weights proved classical distance. In the quantum construction, Boolean evaluation words and signed intersections determine both the encoding and the logical phase.

Backup: affine equivalence changes input coordinates

For m -bit inputs, let A be an invertible binary $m \times m$ matrix and $t \in \mathbb{F}_2^m$ a translation. Matrix operations here use XOR addition. Functions f, g are *input-affine equivalent* if

$$g(u) = f(Au \oplus t).$$

The bijection of input addresses permutes table positions. It preserves weight and, for nonzero functions, degree. Applying the same permutation to two tables preserves their distance. For a direction $v \in \mathbb{F}_2^m$, the derivatives satisfy

$$D_v g(u) = D_{Av} f(Au \oplus t).$$

Permuting physical sites preserves a code and its gate action when every generator word and rotation sign is transported. Translating syndrome entries changes the check data instead.

Backup: the eight-qubit quantum distance is two

The stabilizer checks detect every error acting on one physical qubit. A phase error at one site changes the all-site flip check's outcome to -1 .

Now put phase errors at addresses 000 and 100. Each anticommutes with the all-site flip, so together their two sign changes cancel. The pair has zero syndrome.

The logical word $f_{100,0} = u_1$ is one at 100 and zero at 000. The error pair therefore changes the relative phase of the encoded state labeled by $a = 100$:

$$f_{100,0}(000) \oplus f_{100,0}(100) = 0 \oplus 1 = 1.$$

No one-site error is undetected, but this two-site phase error is undetected and acts logically. The toy code therefore has quantum distance two.

Backup: the 48-qubit logical Boolean functions

On each block w , the function $\kappa_i(u, w)$ evaluates to the logical row K_i . All u_i are bits.

w	row	Boolean function
10	K_1	$u_1 \oplus u_3$
	K_2	$1 \oplus u_1 \oplus u_2 \oplus u_3 \oplus u_4$
	K_3	$1 \oplus u_1 \oplus u_1 u_2 \oplus u_1 u_4 \oplus u_3 u_4$
01	K_1	$1 \oplus u_3$
	K_2	$1 \oplus u_1 \oplus u_3 \oplus u_1 u_3 \oplus u_1 u_4$
	K_3	$u_1 \oplus u_1 u_2 \oplus u_2 u_3 \oplus u_2 u_4 \oplus u_3 u_4$
11	K_1	u_4
	K_2	$u_3 \oplus u_4 \oplus u_1 u_3 \oplus u_1 u_4$
	K_3	$u_1 \oplus u_3 \oplus u_1 u_4 \oplus u_2 u_3 \oplus u_2 u_4$

The current paper's fixed label map. Together with the six coordinate rows, these rows have rank nine.

Backup: the 48-qubit rotation signs

For $u \in \mathbb{F}_2^4$ and $w \in \mathbb{F}_2^2 \setminus \{0\}$, define

$$Q_1(u) = u_1 \oplus u_2 \oplus u_3 \oplus u_1u_3 \oplus u_2u_3 \oplus u_2u_4,$$

$$Q_2(u) = u_3 \oplus u_4 \oplus u_1u_2 \oplus u_1u_4 \oplus u_3u_4.$$

The sign function is

$$\gamma(u, w) = u_3 \oplus w_1Q_1(u) \oplus w_2Q_2(u).$$

Its weights on blocks 10, 01, 11 are 6, 6, 10. Value one selects $T^\dagger$, giving 22 inverse rotations and 26 forward rotations.

The sign polynomial specifies the physical rotations. The fixed logical rows and signed-weight calculation establish the logical gate.

Backup: the scope of the length exclusion

The paper studies CSS codes with three logical qubits and phase distance $d_Z \geq 3$. The class also obeys nine Campbell–Howard parity conditions on individual generator rows and their pair and triple intersections. Within this class, all lengths $n \leq 38$ are excluded, at every stabilizer rank.

physical length	conclusion
$n \leq 38$	no code in this specified class
$39 \leq n \leq 46$	existence remains open
$n = 47$	the particular cited code has no $T/T^\dagger$ -only CCZ pattern
$n = 48$	the constructed code has such a pattern

The explicit 48-qubit construction is not a proof of global minimality. The 47-qubit statement concerns one specified code.

Current paper: length exclusion, fixed-label 47-qubit test, and explicit 48-qubit construction.

Quantum results: the author's current paper, *A 48-qubit native CCZ code from projective geometry and a length lower bound*. The talk uses its fixed logical functions, rotation signs, signed-overlap criterion, and qualified exclusion.

Memory context: IBM, Troubleshooting Memory issues. The eight-bit encoder is a teaching example.

Erasure capacity: Kudekar, Kumar, Mondelli, Pfister, Şaşıoğlu, and Urbanke, Reed–Muller Codes Achieve Capacity on Erasure Channels, Theorem 27.

Boolean fundamentals: Carlet, *Boolean Functions for Cryptography and Coding Theory*. Fourier agreement: O'Donnell, *Analysis of Boolean Functions*, Chapter 1.

Classical coding: MacWilliams–Sloane, *The Theory of Error-Correcting Codes*, and Huffman–Pless, *Fundamentals of Error-Correcting Codes*.